

Lecture 1 - Software Dev Processes

Lecture 2 - Modern Software Engineering

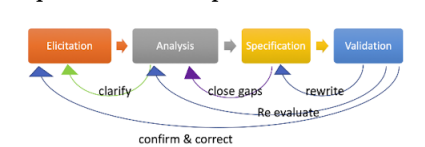
Lecture 3 - Requirements

Requirements are a set of capabilities and constraints [IEEE]

1. A condition or capability needed by a user to solve a problem or achieve an objective
2. A condition or capability that must be met or possessed by a system or system component to satisfy a contract, standard, specification, or other formally imposed documents
3. A documented representation fo a condition or capability as in (1) or (2)

Example of capability: system should allow a requester to specify pick-up and drop-off locations.
Example of constraint: pick-up and drop-off locations should be from a list of on-campus buildings / landmarks.

Requirements Development Phases



Elicitation: Discover requirements (through interview, workshop, document analysis, prototyping)

Analysis: Analyze, decompose, derive, understand, negotiate requirements, identify gaps

Specification: Written and illustrated requirements for comprehension, review, and use

Validation: Confirm correct set of requirements that will enable developers to build a solution

Sources of software requirements

- Documents
 - Descriptions of current or competing products
 - Standards or regulations
 - help desk problem reports
- Interviews
 - Focus groups
 - Use case workshops
 - Product champions as key customer rep
- Questionnaires and marketing surveys
 - Run a pilot first - difficult to ask clear questions
- Event-response tables
 - Identify external stimuli and describe system responses
- Observation - watch users do their jobs
 - Create workflow diagrams
 - "Day in the life" studies
 - Look at what information the user has when performing a particular task
- Prototyping
 - Useful for requirements, design, and implementation

FR vs NFR

FR describes something the system should do

- specify a behavior or function
- e.g. the passenger shall be able to print boarding passes for all flight segments for which she has checked in

NFR describes something not directly related to the functionality of the system

- May describe how well the system works
- Can include other attributes too (e.g. reliability, efficiency, usability)

- E.g. mean time between failure ≥ 9000 hrs

External vs Internal Quality Attributes

External:

- Observed when the software is executing
- Impacts user's experience of using the software
- Develops the user's perception of software quality
- E.g. Availability, performance, robustness, safety, security, reliability, deployability, compatibility, installability, usability, interoperability, integrity

Internal:

- Not directly observed when software is executing
- Perceived by the developers and maintainers
- Encompasses aspects of design that may impact external attributes
- E.g. Scalability, efficiency, modifiability, verifiability, portability, maintainability, reusability, testability

Sample problematic requirements

User service:

- F2 The platform should make sure that trusted users can perform important actions.
 - F2.1 Before a user edits a pickup place, withdraws an errand or marks delivery confirmed, the backend should verify that the user is allowed and return an error code otherwise.
 - F2.2 For every private screen / API call, the edge proxy shall decode the JWT and answer with an error code when teh credential is absent, expired or wrong.

Problems

- F2.1 has points related to supplier svc (editing pickup place) and order svc (withdrawing an errord or marking confirmed)
- Implementation details were included in the requirement (JWT)
- Explain trusted users
- Explain important actions

Supplier service:

- 3. Campus pickup point administration
 - 3.1 The system should display every usable vendor and handover point when a requester starts an errand
 - * 3.1.1 Each result should showw its label and type so users can choose it

Problems:

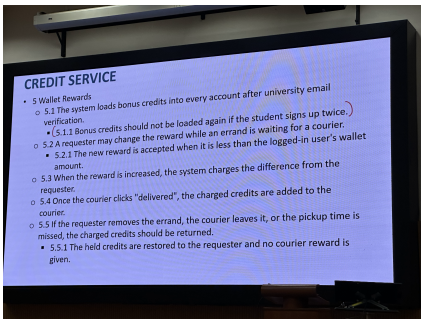
- Missing creating, deleting, editing
- Define "usable"

Supplier service:

- NFR3 Secure supplier administration
 - The system shall use proper permissions for the supplier directory
 - No confidential details should appear when a supplier request fails
 - Extra protction must be added to staff-only supplier URLs

Problems:

- "Proper permissions" is not testable
- Missing indexing



Problems:

- 5.4: Missing necessary condition for credit transfer (requester has to complete)
- 5.1.1: User service should ensure that a student does not sign up twice

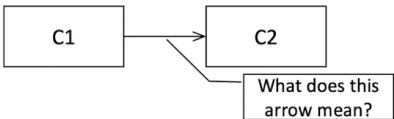
Lecture 4 - Software Architecture

The **Software Architecture** of a program or computing system is the **structure or structures of the system**, which comprise software components, the **externally visible properties** of those components, and the relationships between them. (Externally visible properties = FR + NFR)

In short, the overall organization of the system and a high level design

Building blocks of architecture

- Component: Element that models an application-specific function, responsibility, requirement, task, or process
- Connector: Element that models interactions among components for the purpose of transfer of control and/or data
- Configuration: Topology or structure



Control flow

- Reasoning is on the computation order (computation in C2 depends on computation in C1)

Data flow

- Reasoning is on the data availability (C2 requires data from C1)

Can use solid / dashed lines to differentiate between the 2 flows and add a legend

Message and Event

- Components communiacte via event notifications / message passing / RPC and other protocols
- Async and sync
- Point-to-point

A message is some data sent to a specific address.

- In message-driven systems, each component has a unique address to which other components can send messages
- Each component, or recipient, awaits messages and reacts to them

An event is some data emitted from a component for anyone listening to consume

- An event is emitted by a component when its state changes - record of something that has happened
- They are immutable (cannot be changed or deleted)

- They are ordered in the sequence of their creation
- Sent to the publishing infrastructure, where consumers may later retrieve it

Principle of Modularity

- Modularization → shorter development time, better flexibility, better comprehensibility
- Decomposing a big chunk into smaller chunks with well-defined interfaces (APIs)
- A way to manage the complexity of a problem by breaking them down to smaller manageable modules

Horizontal vs vertical slicing

Horizontal slicing — by technical layer

Boundary follows the technology. One slice = one layer, spanning every domain.

	Users	suppliers	Orders	Credits
Presentation	Profile UI	Vendor list UI	Errand board UI	Wallet UI
API & controllers	/users	/vendors	/orders	/credits
Business logic	Auth & roles	Catalogue rules	Lifecycle & expiry	Reserve & transfer
Persistence	users table	vendors table	orders table	credit ledger

Vertical slicing — by business domain

Boundary follows the domain. One slice = one capability, top to bottom.

	Users	Vendors	Orders	Credits
Presentation	Profile UI	Vendor list UI	Errand board UI	Wallet UI
API & controllers	/users	/vendors	/orders	/credits
Business logic	Auth & roles	Catalogue rules	Lifecycle & expiry	Reserve & transfer
Persistence	users table	vendors table	orders table	credit ledger

Types of cohesion

- Functional
 - Facilities perform only one computation with no side effects
- Layer
 - Related services are kept together, everything else is kept out, and there is a strict hierarchy in which higher-level services can access only lower-level services. Accessing a service may result in side effects
- Communicational
 - Facilities for operating ont he same data are kept together
- Sequential
 - A set of procedures, which work in sequence to perform some computation, are kept together. Output is from one input to the next.
- Procedural
 - A set of procedures, which are called one after another, are kept together
- Temporal
 - Procedures used in the same general phase of execution, such as initialization or termination, are kept together
- Utility
 - Related utilities are kept together, when there is no way to group them using a stronger form of cohesion

Types of coupling

- Content
 - Component modifies internal data of another
- Common / global

- Using global variables; all modules using the global variable are coupled

- Control
 - One module (A) directly controls the other (B) by passing information on what B should do; e.g., using a "flag" that defines what B should do

- Data
 - One module sharing data with another module; e.g., via passing parameters

- External
 - A dependency exists to elements outside the scope of the system, such as the operating system, shared libraries or the hardware.

- Temporal
 - Two actions are bundled together just because they happen to occur at the same time

- Inclusion / import
 - Including a tile or importing a package

Architecture styles

1. How is the code divided?

- Technical concern (horizontal)
 - Focuses on the **separation of concerns**
 - May align with the team's expertise (e.g. frontend, backend, db administrator)
- Domain (business concern) (vertical)
 - Aligned with the domain (business concern)
 - Logical components map to the problem (e.g. customer, payment, shipping)

2. How is the system deployed?

- Single deployed unit (monolithic)
 - Deploy all logical components that make up the application as one unit
 - Application runs as one process
- Distributed units
 - Application consists of independent logical components
 - Logical components run as individual processes
 - Communicate over the network

		Code partitioning	
		Technical	Domain
Code deployment	Monolith	E.g., Layered	E.g., Modular Monolith
	Distributed	E.g., Event-driven	E.g., Microservices